



南開大學
Nankai University

南 開 大 學

人 工 智 能 學 院

实验报告

A 星算法实验

姓名：张浩天

学号：2313607

专业：智能科学与技术

October 27, 2025

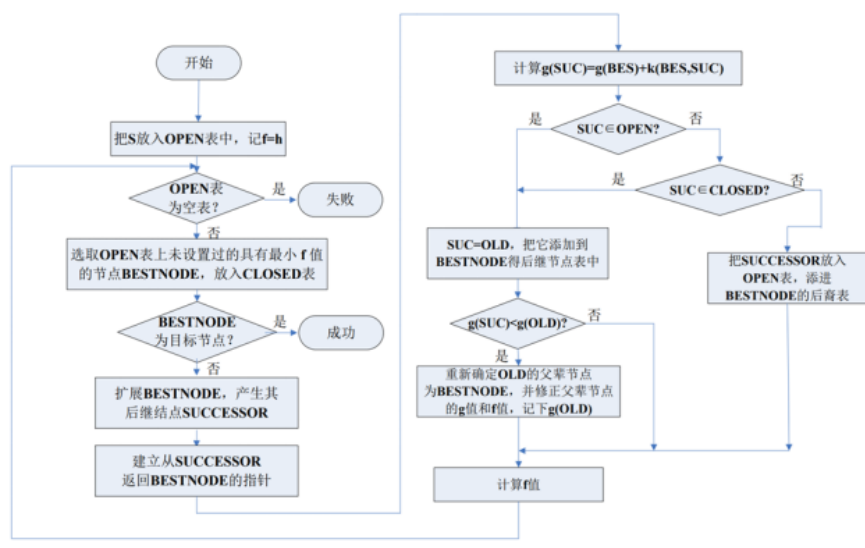
一、 A 星算法实验

(一) 问题描述

A* 算法一种典型的启发式搜索算法，是静态路网中求解最短路径最有效的方法之一。启发式搜索又称为有信息搜索，它是利用问题拥有的启发信息来引导搜索，达到减少搜索范围、降低问题复杂度的目的，通常拥有更好的性能。A* 算法关键部分为启发函数：

$$F = G + H$$

其中，G 为从起点 A 移动到目前方格的代价，H 是该方格到终点 B 的估算成本，F 为寻找下一个遍历节点的依据。



如上流程图，其中，G 为从起点 A 移动到目前方格的代价，H 是该方格到终点 B 的估算成本，F 为寻找下一个遍历节点的依据。

伪代码：

```

1  初始化 open_list 和 close_list ;
2
3  * 将起点加入 open_list 中，并设其移动代价为0；
4
5  * 如果 open_list 不为空，则从 open_list 中选取移动代价最小的节点n：
6      * 如果节点n为终点，则：
7          * 从终点开始逐步追踪 parent 节点，一直达到起点；
8          * 返回找到的结果路径，算法结束；
9
10     * 如果节点n不是终点，则：
11         * 将节点n从 open_list 中移除，并加入 close_list 中；
12         * 遍历节点n所有的邻近节点：
13             * 如果邻近节点m在 close_list 中，则：
14                 * 跳过，选取下一个邻近节点
15             * 如果邻近节点m在 open_list 中，则：
16                 * 计算起点经n到m的g值

```

```
15         * 如果此时计算出来的g值小于原来起点经m的原父节点到m的
           g值:
16             * 更新m的父节点为n, 重新计算m的移动代
               价 f=g+h
17             * 否则, 跳过
18     * 如果邻近节点m不在 open_list 也不在 close_list 中, 则:
19         * 设置节点m的 parent 为节点n
20         * 计算节点m的移动代价 f=g+h
21         * 将节点m加入 open_list 中
```

(二) 实验目的

1. 掌握图搜索算法思路 and 流程, 理解无信息搜索与有信息搜索的区别;
2. 对于启发式搜索, 理解 A* 算法估值函数的选取对算法性能的影响;
3. 对于启发式搜索, 理解 A* 算法求解流程和搜索顺序;
4. 深入理解对图形化界面设计。(选做)

(三) 实验内容

1. 实现 Dijkstra 算法 ($H=0$), 以带有障碍物的二维地图为基础, 寻找到达目标点的最短路径;
2. 实现 A* 算法, 以带有障碍物的二维地图为基础, 寻找到达目标点的最短路径;
3. 对 Dijkstra 算法和 A* 算法的性能进行比较分析;
4. 对实验进行图形化界面设计, 展示搜索过程和最优路径。(选做)

(四) 编程环境

本次实验使用 windows 中的 PyCharm Community Edition 2024.3.4 进行编程, 环境版本为 Python3.12。

(五) 实验步骤

1. 算法实现部分

- (a) 在 A* 算法中, 节点之间的连接关系通过“邻居节点”来表示。每个节点可以与其周围的节点相连, 形成一个图结构。邻居节点的定义在 Spot 类中的 update_neighbors 方法中实现, 具体逻辑如下: 检查当前节点的上下左右四个方向和对角线方向。如果相邻的节点不是障碍物 (通过 is_barrier() 方法判断), 则将其添加到当前节点的邻居列表中, 具体代码如下:

```
1     def update_neighbors(self, grid):
2         self.neighbors = []
3         # 四个基本方向
4         if self.row < self.total_rows - 1 and not grid[self.row + 1][self.col
5             ].is_barrier():
                self.neighbors.append(grid[self.row + 1][self.col])
```

```

6         if self.row > 0 and not grid[self.row - 1][self.col].is_barrier():
7             self.neighbors.append(grid[self.row - 1][self.col])
8         if self.col < self.total_rows - 1 and not grid[self.row][self.col +
9             1].is_barrier():
10             self.neighbors.append(grid[self.row][self.col + 1])
11         if self.col > 0 and not grid[self.row][self.col - 1].is_barrier():
12             self.neighbors.append(grid[self.row][self.col - 1])
13
14         # 对角方向
15         if (self.row < self.total_rows - 1 and self.col < self.total_rows - 1
16             and not
17             grid[self.row + 1][self.col + 1].is_barrier() and not
18             grid[self.row][self.col + 1].is_barrier() and not
19             grid[self.row + 1][self.col].is_barrier()):
20             self.neighbors.append(grid[self.row + 1][self.col + 1]) # 下右
21         if (self.row < self.total_rows - 1 and self.col > 0 and not
22             grid[self.row + 1][self.col - 1].is_barrier() and not
23             grid[self.row + 1][self.col].is_barrier() and not
24             grid[self.row][self.col - 1].is_barrier()):
25             self.neighbors.append(grid[self.row + 1][self.col - 1]) # 下左
26         if (self.row > 0 and self.col < self.total_rows - 1 and not
27             grid[self.row - 1][self.col + 1].is_barrier() and not
28             grid[self.row - 1][self.col].is_barrier() and not
29             grid[self.row][self.col + 1].is_barrier()):
30             self.neighbors.append(grid[self.row - 1][self.col + 1]) # 上右
31         if (self.row > 0 and self.col > 0 and not
32             grid[self.row - 1][self.col - 1].is_barrier() and not
33             grid[self.row - 1][self.col].is_barrier() and not
34             grid[self.row][self.col - 1].is_barrier()):
35             self.neighbors.append(grid[self.row - 1][self.col - 1]) #

```

(b) algorithm 函数是 A* 算法的核心实现，负责执行路径搜索。

- i. 进行初始化。使用优先队列 open_set 存储待探索的节点，并将起始节点添加到队列中。g_score 字典用于记录从起点到每个节点的实际成本，初始值设为无穷大，而起点的被设为 0。
- ii. 在主循环中，算法中获取当前节点并将其标记为已访问。如果当前节点是目标节点，则调用函数重建路径并返回。对于当前节点的每个邻居，算法计算从起点到该邻居的临时 g 值。如果这个新的 g 更小，则更新该邻居的状态，包括其父节点、g、h、f 值，并将其加入优先队列。
- iii. 在计算邻居节点的成本时，斜移动的成本被设置为 14，而直移动的成本为 10，以考虑不同方向的移动成本。同时，通过集合管理当前待探索的节点，确保每个节点仅被添加一次，从而提高算法的效率。

```

1 def algorithm(grid, start, end):
2     count = 0
3     open_set = PriorityQueue() # 优先队列，存储待探索的节点
4     open_set.put((0, count, start))
5     came_from = {}

```

```

6
7     g_score = {spot: float("inf") for row in grid for spot in row}
8     g_score[start] = 0
9
10    open_set_hash = {start}
11    while not open_set.empty():
12        current = open_set.get()[2]
13        open_set_hash.discard(current)
14        current.visited = True
15
16        if current == end:
17            return reconstruct_path(came_from, end)
18
19        for neighbor in current.neighbors:
20            # 计算斜移动的成本
21            temp_g_score = g_score[current] + (14 if abs(neighbor.row -
22                current.row) + abs(neighbor.col - current.col) == 2 else 10)
23
24            if temp_g_score < g_score[neighbor]:
25                came_from[neighbor] = current
26                neighbor.parent = current
27                g_score[neighbor] = temp_g_score
28                neighbor.g = g_score[neighbor]
29                neighbor.h = h(neighbor, end)
30                neighbor.f = neighbor.g + neighbor.h
31
32                if neighbor not in open_set_hash:
33                    count += 1
34                    open_set.put((neighbor.f, count, neighbor))
35                    open_set_hash.add(neighbor)
36
37    return None

```

(c) 可视化部分

- i. 函数 `algorithm_visual` 在每次发现新邻居或弹出一个节点时都会刷新画面，并在搜索过程中把所有被发现或已弹出的格子显示 `ghf` 值并临时变为白色；找到终点后把路径染成路径颜色。

```

1 def algorithm_visual(win, grid, start, end, clock, use_dijkstra):
2     count = 0
3     open_set = PriorityQueue() # 优先队列，存储待探索的节点
4     open_set.put((0, count, start))
5     came_from = {} # 记录每个节点的父节点
6
7     # 存储节点到起点的实际代价
8     g_score = {spot: float("inf") for row in grid for spot in row}
9     g_score[start] = 0
10    start.g = 0

```

```
11     start.h = h(start, end)
12     start.f = start.g + start.h
13
14     # 标记起点为已发现 (加入 open set)
15     start.discovered = True
16     close_list = set()
17
18     print("开始循环 (可视化)")
19     # 当队列不为空时循环
20     while not open_set.empty():
21         # 处理 pygame 事件避免界面无响应
22         for ev in pygame.event.get():
23             if ev.type == pygame.QUIT:
24                 pygame.quit()
25                 return None
26
27         current = open_set.get()[2] # 取出当前节点 (最低 f)
28         # 标记为已弹出并处理
29         current.visited = True
30         current.discovered = True
31         close_list.add(current)
32
33         if current != start and current != end:
34             current.color = 'white'
35
36         # 每弹出一个节点就绘制一次 (可视化弹出)
37         win.fill((0, 0, 0))
38         draw_grid(win, grid)
39         draw(win, grid, close_list)
40         pygame.display.update()
41         # 延迟以便观察 (毫秒), 可调整使搜索更快或更慢
42         pygame.time.delay(30)
43         clock.tick(60)
44
45         if current == end:
46             # 找到终点, 重构路径并返回
47             path = reconstruct_path(came_from, end)
48             # 将路径格标记为 in_path 并改变颜色为路径颜色
49             for spot in path:
50                 spot.in_path = True
51                 # 不覆盖起点/终点颜色
52                 if spot.color not in ('green', 'red'):
53                     spot.color = 'purple'
54             # 最后一次绘制以显示完整路径
55             win.fill((0, 0, 0))
56             draw_grid(win, grid)
57             draw(win, grid, close_list)
58             pygame.display.update()
```

```
59         return path
60
61     for neighbor in current.neighbors:
62         # 处理 pygame 事件以便在长时间搜索时仍可响应
63         for ev in pygame.event.get():
64             if ev.type == pygame.QUIT:
65                 pygame.quit()
66                 return None
67
68         # 对角移动的成本为 14，其余为 10（与 h 同单位）
69         if abs(neighbor.row - current.row) + abs(neighbor.col - current.col) == 2:
70             temp_g_score = g_score[current] + 14
71         else:
72             temp_g_score = g_score[current] + 10
73
74         # 当发现更短路径时更新
75         if temp_g_score < g_score[neighbor]:
76             came_from[neighbor] = current
77             neighbor.parent = current
78             g_score[neighbor] = temp_g_score
79
80         # 更新邻居节点的 g 值
81         neighbor.g = g_score[neighbor]
82
83         # 如果使用 A*, 计算 f 值
84         if not use_dijkstra:
85             neighbor.h = h(neighbor, end)
86             neighbor.f = neighbor.g + neighbor.h
87         else:
88             neighbor.f = neighbor.g # Dijkstra 只使用 g
89
90         # 把邻居标记为已发现（加入 open set）
91         neighbor.discovered = True
92         # 在搜索过程中把被发现的格子显示为白色
93         if neighbor.color not in ('green', 'red'):
94             neighbor.color = 'white'
95
96         count += 1
97         open_set.put((neighbor.f, count, neighbor))
98
99         # 每发现一个邻居就绘制一次（便于观察 frontier 的扩展）
100        win.fill((0, 0, 0))
101        draw_grid(win, grid)
102        draw(win, grid, close_list)
103        pygame.display.update()
104        pygame.time.delay(20)
105        clock.tick(60)
```

```

106
107     # 如果没有找到路径
108     return None

```

ii. 函数 `reconstruct_path` 构建从起点到终点的路径。

```

1 def reconstruct_path(came_from, current):
2     path = []
3     while current in came_from:
4         path.append(current)
5         current = came_from[current]
6     # 添加起点
7     path.append(current)
8     return path[::-1]

```

iii. 函数 `draw` 根据网格状态绘制每个节点的颜色、指向父节点的指针和 `g`、`h`、`f` 的值。

```

1 def draw(win, grid, close_list):
2     COLOR_MAP = {
3         'black': (0, 0, 0),
4         'blue': (0, 0, 255),
5         'green': (0, 255, 0),
6         'red': (255, 0, 0),
7         'yellow': (255, 255, 0),
8         'purple': (160, 32, 240),
9         'white': (255, 255, 255)
10    }
11    # 绘制指针与数值（在所有格子颜色后绘制）
12    for row in grid:
13        for spot in row:
14            # 绘制指向父节点的小指针（只对已被发现或已弹出的格子绘制，并且有
            # parent）
15            if (spot.discovered or spot.in_path or spot.visited) and spot.
                parent is not None:
16                cx = spot.x + spot.width / 2
17                cy = spot.y + spot.width / 2
18                pcx = spot.parent.x + spot.parent.width / 2
19                pcy = spot.parent.y + spot.parent.width / 2
20                # 指针颜色选择：浅灰色
21                draw_arrow(win, (cx, cy), ((pcx-(pcx-cx)*0.7), (pcy-(pcy-cy)
                    *0.7)), color=(0, 0, 0),
22                           arrow_size=max(4, spot.width // 12), width=2)
23            # 给被关闭的格子（visited）绘制蓝色外框，便于区分
24            if spot.visited:
25                pygame.draw.rect(win, (0, 0, 255), (spot.x, spot.y, spot.
                    width, spot.width), 2)
26
27            # 绘制 ghf 数值：仅当被发现/已弹出/属于路径时显示
28            if spot.discovered or spot.visited or spot.in_path:

```



```

29         # f 放左上, g 放左下, h 放右下
30         padding = max(2, spot.width // 20)
31         # 使用 FONT 渲染文本
32         if FONT is None:
33             fnt = pygame.font.SysFont(None, max(12, spot.width // 5))
34         else:
35             # 当 FONT 的大小与 cell 相差较大时, 为了保证可读性按 cell
               动态选择大小
36             fnt = pygame.font.SysFont(None, max(10, spot.width // 4))
37         # 显示 g (左下)
38         g_text = '∞' if spot.g == float("inf") else str(int(spot.g))
39         g_surf = fnt.render(g_text, True, (0, 0, 0))
40         gx = spot.x + padding
41         gy = spot.y + spot.width - g_surf.get_height() - padding
42         # 显示 h (右下)
43         h_text = '∞' if spot.h == float("inf") else str(int(spot.h))
44         h_surf = fnt.render(h_text, True, (0, 0, 0))
45         hx = spot.x + spot.width - h_surf.get_width() - padding
46         hy = spot.y + spot.width - h_surf.get_height() - padding
47         # 显示 f (左上)
48         f_text = '∞' if spot.f == float("inf") else str(int(spot.f))
49         f_surf = fnt.render(f_text, True, (0, 0, 0))
50         fx = spot.x + padding
51         fy = spot.y + padding
52
53         # 在深色背景上使用白色字体以提高可见性: 判断颜色亮度
54         bg = COLOR_MAP.get(spot.color, spot.color)
55         if isinstance(bg, tuple):
56             brightness = (bg[0] * 0.299 + bg[1] * 0.587 + bg[2] *
               0.114)
57         else:
58             brightness = 255
59         text_color = (0, 0, 0) if brightness > 160 else (255, 255,
               255)
60         # 重新渲染带颜色的文本
61         g_surf = fnt.render(g_text, True, text_color)
62         h_surf = fnt.render(h_text, True, text_color)
63         f_surf = fnt.render(f_text, True, text_color)
64
65         win.blit(f_surf, (fx, fy))
66         win.blit(g_surf, (gx, gy))
67         win.blit(h_surf, (hx, hy))

```

iv. 函数 `draw_grid` 绘制整个网格的结构: 包括网格中的小圈和网格的白色边界线。

```

1 def draw_grid(win, grid):
2     COLOR_MAP = {
3         'black': (0, 0, 0),
4         'blue': (0, 0, 255),

```

```

5         'green': (0, 255, 0),
6         'red': (255, 0, 0),
7         'yellow': (255, 255, 0),
8         'purple': (160, 32, 240),
9         'white': (255, 255, 255)
10    }
11    # 如果 FONT 未被初始化, 使用默认小号字体
12    global FONT
13    for row in grid:
14        for spot in row:
15            # 绘制格子颜色 (搜索过程中 discovered/visited 的格子会被设置为 '
                white')
16            color = COLOR_MAP.get(spot.color, spot.color)
17            pygame.draw.rect(win, color, (spot.x, spot.y, spot.width, spot.
                width))
18
19            # 在方格中间绘制小圆, 小圆的半径为方格大小的 1/10 (用于视觉参考)
20            circle_radius = max(1, spot.width // 8)
21            circle_center = (spot.x + spot.width // 2, spot.y + spot.width //
                2)
22            pygame.draw.circle(win, (255, 165, 0), circle_center,
                circle_radius, width=2) # 白色空心圆
23
24    # 绘制白色实线边界 (确保网格分隔线在最上层)
25    for row in grid:
26        for spot in row:
27            pygame.draw.line(win, (255, 255, 255), (spot.x, spot.y), (spot.x
                + spot.width, spot.y)) # 上边
28            pygame.draw.line(win, (255, 255, 255), (spot.x, spot.y), (spot.x,
                spot.y + spot.width)) # 左边
29            pygame.draw.line(win, (255, 255, 255), (spot.x + spot.width, spot
                .y),
30                                (spot.x + spot.width, spot.y + spot.width)) #
                右边
31            pygame.draw.line(win, (255, 255, 255), (spot.x, spot.y + spot.
                width),
32                                (spot.x + spot.width, spot.y + spot.width)) #
                下边

```

v. 函数 draw_path 将找到的路径设置为紫色

```

1 def draw_path(win, path, grid):
2     for spot in path:
3         spot.in_path = True
4         # 避免把起点/终点覆盖为紫色 (但仍显示路径标识), 保留起点/终点颜色可
            选:
5         if spot.color not in ('green', 'red'):
6             spot.color = 'purple' # 路径颜色
7     # 不在此处重复绘制格子, 主循环会调用 draw_grid 来统一渲染

```

- vi. 主函数 `main()` 是整个程序的入口，初始化 Pygame 窗口、设置网格、处理输入以及运行 A 星算法的可视化。实现了通过鼠标点击和键盘输入与程序进行交互，包括设置起点、终点、墙体、运行算法和清空网格的功能。

```

1 def main():
2     # 使用 tkinter 弹窗获取网格大小 n
3     root = tk.Tk()
4     root.withdraw()
5     n = simplifiedialog.askinteger("Grid size", "请输入网格行/列数 n (例如 7):",
6                                   minvalue=3, maxvalue=100)
7     root.destroy()
8     if n is None:
9         print("未输入网格大小，程序退出。")
10        return
11
12    pygame.init()
13    width = 750
14    rows = n
15    win = pygame.display.set_mode((width, width))
16    pygame.display.set_caption("A* 可视化 (左键: 设置 起点/终点/墙; 右键: 清除; 空格: 运行; C: 清空)")
17    # 初始化字体 (相对于单元格大小)
18    global FONT
19    gap = width // rows
20    FONT = pygame.font.SysFont(None, max(12, gap // 4))
21
22    grid = make_grid(rows, width)
23
24    # 状态变量
25    start = None
26    end = None
27    path = []
28    running_algo = False
29
30    # 在运行算法前确保每个点的 neighbors 已更新 (当无障碍时，此处也可初始化)
31    for row in grid:
32        for spot in row:
33            spot.update_neighbors(grid)
34
35    run = True
36    clock = pygame.time.Clock()
37
38    while run:
39        clock.tick(60)
40        # 在每一帧清屏为黑色背景，然后绘制网格 (网格函数会根据 spot.color 绘制每个格子)
41        win.fill((0, 0, 0))
42        draw_grid(win, grid)
43        close_list = set()

```

```

43     draw(win, grid, close_list)
44
45     # 在顶部绘制提示信息
46     instr_font = pygame.font.SysFont(None, 20)
47     if start is None:
48         instr = "左键：设置 起点 (A)"
49     elif end is None:
50         instr = "左键：设置 终点 (T)"
51     else:
52         instr = "左键：放置墙体；右键：删除方块；按 空格 运行算法；按 C
                    清空所有设置"
53     instr_surf = instr_font.render(instr, True, (255, 255, 255))
54     win.blit(instr_surf, (10, 10))
55
56     # 如果已经有起点和终点，显示它们的坐标
57     info = ""
58     if start:
59         info += f"Start: ({start.row},{start.col}) "
60     if end:
61         info += f"End: ({end.row},{end.col})"
62     info_surf = instr_font.render(info, True, (255, 255, 255))
63     win.blit(info_surf, (10, 30))
64
65     pygame.display.update()
66
67     for event in pygame.event.get():
68         if event.type == pygame.QUIT:
69             run = False
70
71         if event.type == pygame.MOUSEBUTTONDOWN:
72             pos = pygame.mouse.get_pos()
73             row, col = get_clicked_pos(pos, rows, width)
74             spot = grid[row][col]
75
76             if event.button == 1: # 左键
77                 # 如果还没有设置起点，设置起点
78                 if start is None and spot != end:
79                     start = spot
80                     start.color = 'green'
81                     start.g = 0
82                     start.f = 0
83                     start.parent = None
84                     start.visited = False
85                     start.in_path = False
86                     start.discovered = True
87                 # 如果已经有起点但没有终点，设置终点
88                 elif start is not None and end is None and spot != start:
89                     end = spot

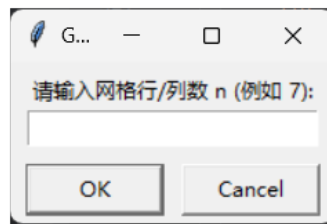
```

```
90         end.color = 'red'
91         end.parent = None
92         end.visited = False
93         end.in_path = False
94         end.discovered = False
95     # 如果都已设置, 则左键切换为墙体 (排除起点与终点)
96     elif spot != start and spot != end:
97         if not spot.is_barrier():
98             spot.color = 'blue'
99         else:
100             spot.color = 'black'
101
102     # 每次修改后更新所有邻居 (简单粗暴)
103     for r in grid:
104         for s in r:
105             s.update_neighbors(grid)
106
107     elif event.button == 3: # 右键: 清除该格 (如果是起点或终点则
108         # 清除相应引用)
109         if spot == start:
110             start.reset()
111             start = None
112         elif spot == end:
113             end.reset()
114             end = None
115         else:
116             spot.reset()
117
118     # 更新邻居
119     for r in grid:
120         for s in r:
121             s.update_neighbors(grid)
122
123     if event.type == pygame.KEYDOWN:
124         if event.key == pygame.K_SPACE:
125             # 运行算法 (需要先有 start 和 end)
126             if start is None or end is None:
127                 print("请先设置起点和终点。")
128                 continue
129
130             # 在开始算法前, 清除之前的 visited/in_path/parent/g/h/f/
131             # discovered (如果需要重新搜索)
132             for row in grid:
133                 for spot in row:
134                     # 保留墙体、起点、终点颜色, 重置其它状态
135                     if spot not in (start, end) and not spot.is_barrier():
136                         spot.color = 'black'
```

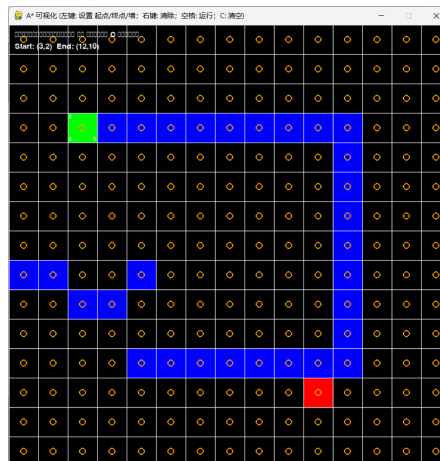
```
135         spot.visited = False
136         spot.in_path = False
137         spot.parent = None
138         spot.g = float("inf")
139         spot.h = 0
140         spot.f = float("inf")
141         spot.discovered = False
142     start.g = 0
143     start.h = h(start, end)
144     start.f = start.g + start.h
145     start.discovered = True
146     start.parent = None
147
148     # 更新所有邻居 (确保最新障碍关系)
149     for r in grid:
150         for s in r:
151             s.update_neighbors(grid)
152
153     # 以可视化方式运行算法 (algorithm_visual 内部会刷新屏幕)
154     path = algorithm_visual(win, grid, start, end, clock,
155                             use_dijkstra=False)
156     if path:
157         draw_path(win, path, grid)
158         # 确保路径节点也被标记为 visited (方便显示 ghf)
159         for p in path:
160             p.visited = True
161             p.discovered = True
162         pygame.display.update()
163         print("找到路径, 已显示。")
164     else:
165         print("未找到路径")
166
167     if event.key == pygame.K_c:
168         # 清空所有设置, 支持重复设置并重新运行
169         start = None
170         end = None
171         path = []
172         for row in grid:
173             for spot in row:
174                 spot.reset()
175         # 更新邻居
176         for r in grid:
177             for s in r:
178                 s.update_neighbors(grid)
179         print("已清空网格, 您可以重新设置起点、终点和墙体。")
180
181     pygame.quit()
```

(六) 实验结果

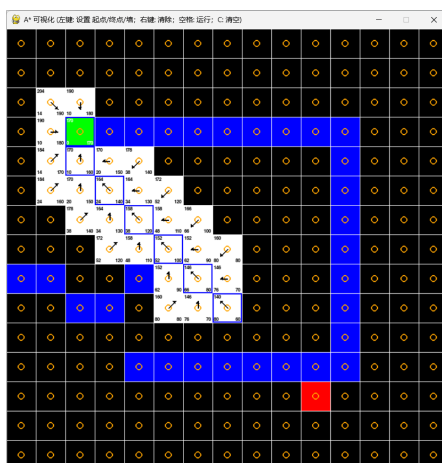
1. 程序运行开始之后，会出现一个窗口可以输入 n 的值，也就是格子的大小。



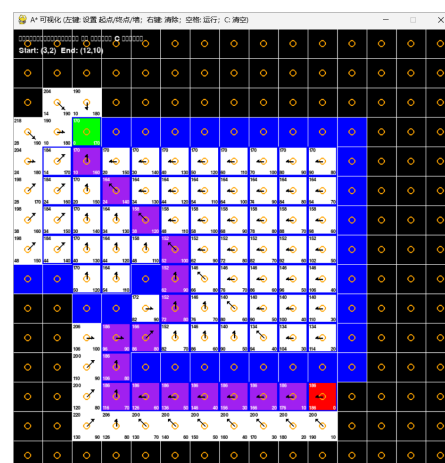
2. 输入 n 的大小点击运行之后，使用鼠标左键可以设置起点、终点、墙体，点击空格之后开始寻找路径，点击鼠标右键可以清除单个格子的状态，点击 C 键可以清除全图



设置好起点、终点和墙体

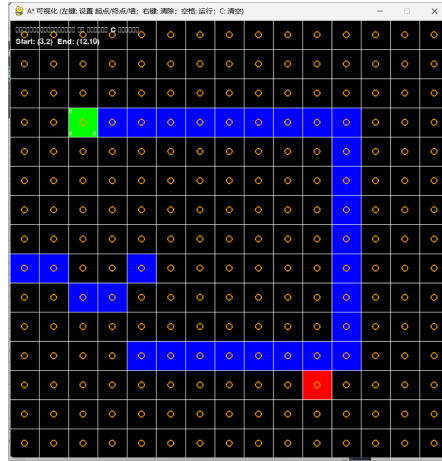


路径寻找过程中



已经寻找到路径

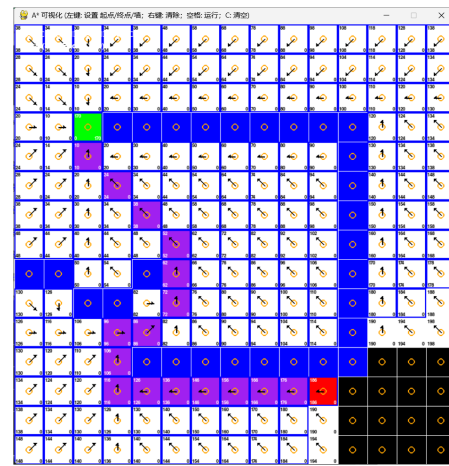
使用 A 星算法, $n=15$



设置好起点、终点和墙体



路径寻找过程中



已经寻找到路径

使用 dijkstra 算法, $n=15$

(七) 分析总结

针对墙角是否可以穿越对比分析：

1. 本次实验中，根据老师要求实现的是墙角不可以通过对角线通过，在代码中，可以通过定义邻居节点中设置，这部分的代码如下：

```

1 # 对角方向
2 if (self.row < self.total_rows - 1 and self.col < self.total_rows - 1
    and not
3     grid[self.row + 1][self.col + 1].is_barrier() and not
4     grid[self.row][self.col + 1].is_barrier() and not
5     grid[self.row + 1][self.col].is_barrier()):
6     self.neighbors.append(grid[self.row + 1][self.col + 1]) # 下右
7 if (self.row < self.total_rows - 1 and self.col > 0 and not
8     grid[self.row + 1][self.col - 1].is_barrier() and not
9     grid[self.row + 1][self.col].is_barrier() and not

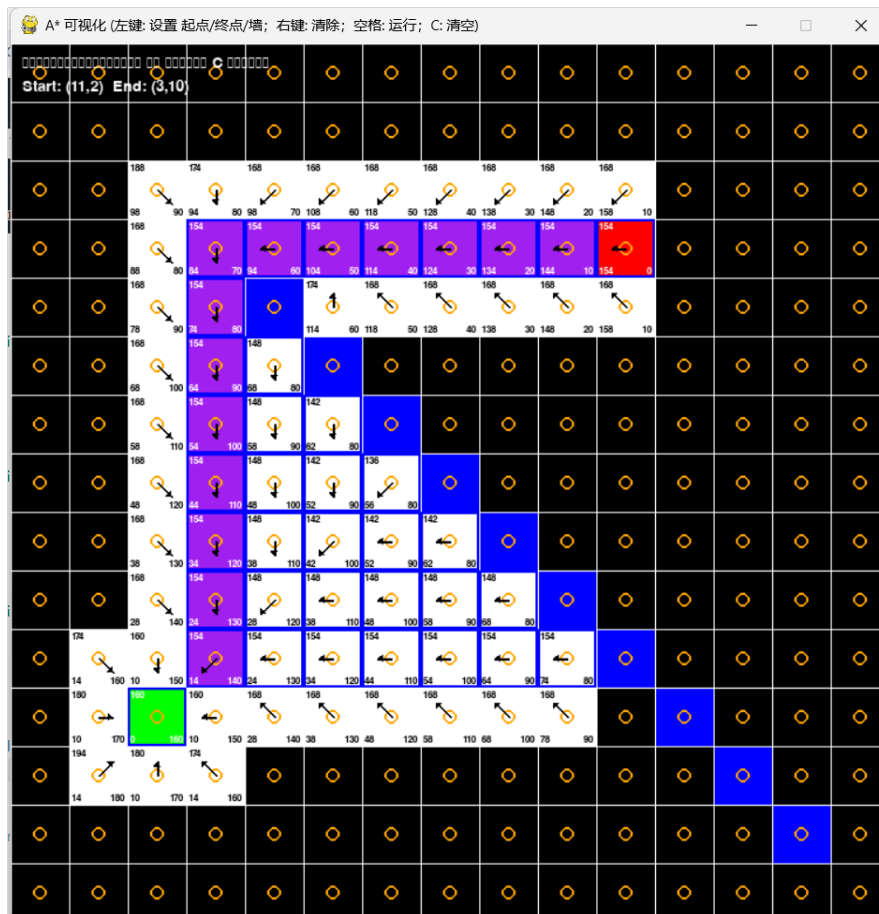
```



```

10         grid[self.row][self.col - 1].is_barrier()):
11         self.neighbors.append(grid[self.row + 1][self.col - 1]) # 下左
12         if (self.row > 0 and self.col < self.total_rows - 1 and not
13             grid[self.row - 1][self.col + 1].is_barrier() and not
14             grid[self.row - 1][self.col].is_barrier() and not
15             grid[self.row][self.col + 1].is_barrier()):
16         self.neighbors.append(grid[self.row - 1][self.col + 1]) # 上右
17         if (self.row > 0 and self.col > 0 and not
18             grid[self.row - 1][self.col - 1].is_barrier() and not
19             grid[self.row - 1][self.col].is_barrier() and not
20             grid[self.row][self.col - 1].is_barrier()):
21         self.neighbors.append(grid[self.row - 1][self.col - 1]) # 上左

```



墙角不可以对角线通过

2. 本次实验中，根据老师要求实现的是墙角不可以通过对角线通过，在代码中，可以通过定义邻居节点中设置，这部分的代码如下：

```

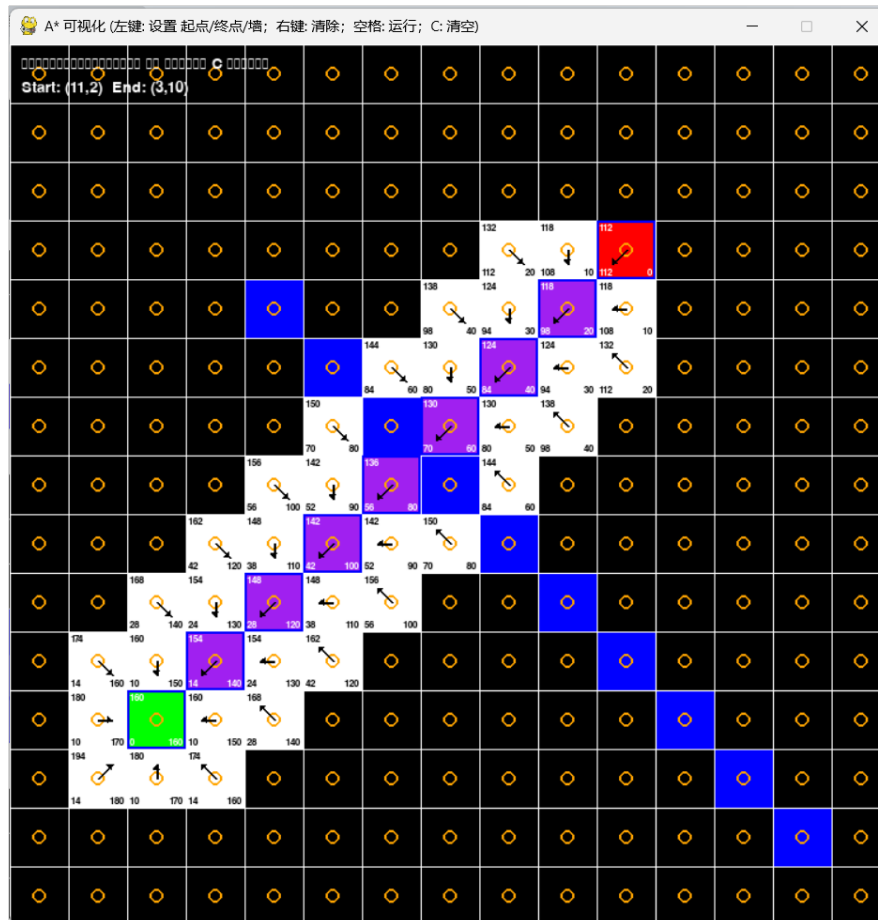
1     # 对角方向
2     if (self.row < self.total_rows - 1 and self.col < self.total_rows - 1
3         and not
4         grid[self.row + 1][self.col + 1].is_barrier()):
5         self.neighbors.append(grid[self.row + 1][self.col + 1])

```

```

5         if (self.row < self.total_rows - 1 and self.col > 0 and not
6             grid[self.row + 1][self.col - 1].is_barrier()):
7             self.neighbors.append(grid[self.row + 1][self.col - 1])
8         if (self.row > 0 and self.col < self.total_rows - 1 and not
9             grid[self.row - 1][self.col + 1].is_barrier()):
10            self.neighbors.append(grid[self.row - 1][self.col + 1])
11        if (self.row > 0 and self.col > 0 and not
12            grid[self.row - 1][self.col - 1].is_barrier()):
13            self.neighbors.append(grid[self.row - 1][self.col - 1])

```



墙角可以对角线通过

由上面代码以及图像显示可知，允许对角线通行的实现通常会得到更优的路径，尤其是在网格中有多墙体的情况下，这种实现能够有效减少路径长度。尽管允许对角线通行可能导致算法探索的邻居更多，但由于路径更短，整体运行时间往往得到改善。